CHIPEDGE · ENGINEERING AUTOMATION PLATFORM

Course Materials Storage & Delivery Specification

Free high-capacity storage for documents, video and audio - and the code changes to get there

Supabase's free tier is not the real constraint. The application currently stores file bytes inside PostgreSQL rows, duplicated once per recipient, and writes everything else to a disk that Render erases on every deploy. This document sizes the true requirement, compares the free storage platforms worth using, and specifies exactly what changes in the schema, the API and the code - including how materials plug into the one-trigger automation platform.

THE SHORT ANSWER

- Roughly 66 GB of free object storage plus unlimited free video hosting is available today.
- Cloudflare R2 is the primary recommendation - 10 GB free and, uniquely, zero egress charges.
- The biggest win is not a new provider: stop storing base64 in Postgres and stop attaching files to mail.
- Materials are per-COURSE, not per-batch. Store once, link many - that is what keeps you inside free tiers.

REPOSITORY	engg-automation-git-file · branch rollback-stable
COMPANION	AUTOMATION_BLUEPRINT.pdf (one-trigger orchestration)
SCOPE	Read-only analysis. No application code was modified.
PREPARED	04 August 2026

Contents

1. Executive Summary

The request was for free platforms with large storage. That is answered in Section 4. But the analysis found something more urgent: the application is consuming its scarcest resource - the PostgreSQL database - to store file bytes, and multiplying them by the number of recipients.

The finding that matters most

> **CRITICAL ATTACHMENTS ARE BASE64-ENCODED INTO A DATABASE ROW, ONCE PER LEARNER**
>
> backend/index.js:9478 reads an uploaded file and writes fileBuffer.toString("base64") into the scheduled_emails.attachment_data column, inside a loop over every learner in the batch. Base64 inflates a file by about 37%. A single 5 MB mock-interview feedback PDF sent to 25 learners therefore writes roughly 170 MB into Postgres. Three such sends can exhaust a 500 MB free database - and none of those bytes ever needed to be in the database at all.

This single pattern explains most of the pressure on the Supabase quota. Fixing it costs less than moving providers and frees more space than any free tier grants.

What the fix looks like

- **Bytes never enter Postgres.** The database stores metadata and an object key - typically under 300 bytes per material - never the file itself.
- **Bytes never touch the Render dyno.** The browser uploads directly to object storage using a pre-signed URL. This removes the 10 MB body limit, the request timeout ceiling and the ephemeral-disk problem in one change.
- **Email carries links, not attachments.** A signed, expiring link is a few hundred bytes and works for a 2 GB video. Mailjet caps a whole message at about 15 MB, so attachments were never a viable path for real course material.
- **Materials are stored once per course, not once per batch.** Content-addressed keys mean the same lecture video shared by six batches occupies storage once.

Capacity available at zero cost

PLATFORM	FREE STORAGE	FREE EGRESS	ROLE IN THE DESIGN
Cloudflare R2	10 GB	Unlimited	Primary - video, audio, large docs
Storj DCS	25 GB	25 GB / month	Secondary + archive tier
Backblaze B2	10 GB	3x storage / month	Cold backup
Google Drive	15 GB / account	Generous	Bulk documents
Firebase Storage	5 GB	1 GB / day	Already credentialed - see 4.4
Supabase Storage	1 GB	Limited	Small hot documents only
YouTube (unlisted)	Unlimited	Unlimited	Lecture video delivery + CDN

Approximately 66 GB of object storage plus unlimited video hosting, at no cost. Figures are as published by each vendor and must be re-checked before you commit - free tiers change, and Section 12 explains why the architecture is deliberately provider-agnostic so that a change costs one adapter file.

Why egress, not storage, is the real constraint

Storage is cheap and easy to find. Bandwidth is what generates surprise bills. Twenty-five learners each streaming a 15 GB course library is 375 GB of egress per batch. On most providers that is the expensive line; on Cloudflare R2 it is free, which is the single strongest reason R2 is the primary recommendation rather than a larger-but-metered alternative.

2. How the Code Handles Files Today

Nine findings, all verified against the current branch. Together they explain the storage pressure and define the work in Section 8.

#	FINDING	LOCATION	IMPACT
F1	Attachments base64-encoded into Postgres, per recipient	index.js:9470-9489	Critical
F2	multer({ dest: "uploads/" }) - no size limit, no MIME filter	index.js:137	High
F3	Public unauthenticated file host via /api/upload + static /uploads	index.js:6938, 6950	High
F4	Upload URLs break on every redeploy (ephemeral disk)	index.js:6942	High
F5	Supabase Storage never used anywhere in the codebase	repo-wide grep: no matches	Opportunity
F6	Firebase Admin already initialised with a service account	push/firebaseAdmin.js:56	Opportunity
F7	Upload artefacts committed to git as a workaround	backend/uploads/ (8 files)	Symptom
F8	express.json limit of 10 MB caps any base64 upload path	index.js:126	Medium
F9	email-worker.js reads misspelled attachement_* columns	email-worker.js:75-79	Latent bug

2.1 The base64 loop

```
// backend/index.js:9462  - POST /api/mock-interview-feedback
const fileBuffer = fs.readFileSync(file.path);

for (const learner of learners) {                      // <- once PER LEARNER
  await supabase.from("scheduled_emails").insert({
    recipient_email: learner.email,
    attachment_name: originalName,
    attachment_data: fileBuffer.toString("base64"),    // <- +37% size, in Postgres
    ...
  });
}

//  5 MB PDF  ->  6.85 MB base64  x  25 learners  =  ~171 MB of database
//  ...for ONE send. The bytes are identical in every row.
```

The same pattern would appear anywhere attachments are added later. The remedy in Section 7 replaces attachment_data with a nullable attachment_material_id foreign key - eight bytes instead of seven megabytes - and lets the dispatcher decide at send time whether to attach a small file or insert a signed link.

2.2 The public file host

```
// backend/index.js:6938
app.post('/api/upload', upload.single('file'), (req, res) => {
  const fileUrl = `${req.protocol}://${req.get('host')}/uploads/${req.file.filename}`;
  res.json({ url: fileUrl });                 // no auth, no validation, no size cap
});

app.use('/uploads', express.static('uploads'));   // serves everything, to anyone
```

CRITICAL THREE PROBLEMS IN SEVEN LINES

There is no authentication, so any anonymous caller can upload any file of any size and have it served back from your domain. There is no MIME or extension check, so the endpoint will happily host active content. And the returned URL points at a directory Render deletes on the next deploy, so every link ever handed out eventually 404s. This endpoint should be removed rather than fixed - Section 7 replaces it with the pre-signed flow.

2.3 What is already in your favour

- **Firebase Admin is live.** push/firebaseAdmin.js already initialises the SDK from a service account for FCM. The frontend config names project ce0001 and bucket ce0001.firebasestorage.app. Google Cloud Storage is therefore reachable with admin.storage() and no new credentials - see the caveat in 4.4.
- **The dispatcher is a good integration point.** Every outbound message already flows through one per-minute cron over scheduled_emails. Converting attachments to links is a change in one place, not thirty.
- **Nothing depends on Supabase Storage yet.** Because it was never adopted, there is no migration debt. The adapter can be introduced cleanly.

3. Sizing the Real Requirement

Free tiers only work if the volume is understood. These figures are modelled on a 12-week programme with three domains (PD, DV, DFT) and roughly 25 learners per batch.

3.1 Per-course library

ASSET CLASS	TYPICAL VOLUME	UNIT SIZE	SUBTOTAL	GROWS WITH
Slides / PDFs	120-200 files	2-8 MB	0.6-1.2 GB	Course version
Lab manuals, datasheets	60-100 files	1-5 MB	0.2-0.4 GB	Course version
Reference scripts / RTL	200-400 files	10-500 KB	under 0.1 GB	Course version
Lecture video (720p H.264)	55-70 sessions	180-260 MB/hr	12-18 GB	Course version
Audio-only revision tracks	55-70 tracks	35-45 MB/hr	1.5-2.5 GB	Course version
Recorded doubt sessions	10-20 per batch	150-250 MB	2-4 GB	Per batch

A single domain's complete library is therefore roughly 15-22 GB, dominated by video. Three domains give a steady state near 45-60 GB, growing by a few GB per year as content is revised.

3.2 The insight that keeps you inside free tiers

> **RECOMMENDED STORE PER COURSE VERSION, LINK PER BATCH**
>
> The lecture library for PD 2026 is identical for every PD batch that runs it. If materials are keyed by batch, ten batches store the same 15 GB ten times - 150 GB, and no free tier survives that. If materials are keyed by course version and merely LINKED to batches, the same ten batches consume 15 GB once. This single modelling decision is worth more than any provider choice, and it is why Section 6 separates the materials table from the material_links table.

3.3 Bandwidth, the line item people forget

SCENARIO	EGRESS	ON R2 (FREE EGRESS)	ON A METERED PROVIDER
One learner downloads the full library	15 GB	0.00	approx. 1.35 USD at 0.09/GB
One batch of 25 learners	375 GB	0.00	approx. 34 USD
Six batches per year	2.25 TB	0.00	approx. 202 USD
Same, but video served by YouTube	under 50 GB	0.00	approx. 4 USD

Storage for this workload costs a few dollars a month anywhere. Egress is what turns a hobby bill into a real one, and it is the reason the recommended stack pairs zero-egress object storage with a free video CDN.

4. Free Storage Platforms - Comparison

> **CAUTION VERIFY EVERY FIGURE BEFORE YOU COMMIT**
> Free-tier terms change without notice, and several providers have tightened them in recent years. The numbers below are as published by each vendor and are given to rank the options, not as a guarantee. The architecture in Section 5 deliberately hides the provider behind an adapter so that switching later is a one-file change rather than a migration.

4.1 Object storage (S3-compatible)

PROVIDER	FREE STORAGE	FREE EGRESS	API	VERDICT
Cloudflare R2	10 GB	Unlimited	S3	Best overall. Zero egress is unique and decisive for video. Class A/B operation limits are generous for this workload.
Storj DCS	25 GB	25 GB / month	S3	Largest free storage. Decentralised, so first-byte latency is higher. Excellent as the archive tier.
Backblaze B2	10 GB	3x stored / month	S3	Solid and simple. Egress to Cloudflare is free via the Bandwidth Alliance, which pairs well with R2.
Firebase / GCS	5 GB	1 GB / day	GCS SDK	Credentials already exist in this project. See 4.4 for an important billing caveat.
Supabase Storage	1 GB	Limited	Supabase	Too small for media. Keep for small, hot documents where you want one system.
AWS S3	5 GB	100 GB / month	S3	Not recommended - free tier expires after 12 months, then bills silently.
Azure Blob	5 GB	100 GB / month	Azure	Same 12-month expiry problem.

4.2 Media-specific platforms

PLATFORM	FREE ALLOWANCE	STRENGTH	LIMITATION
YouTube (unlisted)	Unlimited video	Global CDN, adaptive bitrate, zero cost, mobile-friendly player	Unlisted is obscurity, not access control. Anyone with the link can view and share it.
Cloudinary	~25 credits / month	Automatic transcoding, thumbnails, adaptive streaming	Credit model conflates storage, bandwidth and transforms - easy to exhaust
Bunny Stream	Not free (approx. 0.005 USD/GB stored)	Purpose-built video, very cheap, token-authenticated playback	Paid, but likely under 5 USD/month at this scale
Cloudflare Stream	Not free	Signed playback URLs, per-minute pricing, real DRM-adjacent control	Costs scale with watch time
Vimeo Free	5 GB total	Clean player, no ads	Far too small for a course library
Internet Archive	Unlimited	Genuinely free forever	Public by design - unsuitable for proprietary material

4.3 Documents at bulk

Google Drive gives 15 GB per free account and has a mature API with resumable uploads and per-file sharing. It is a reasonable overflow tier for documents, and a Google Workspace account (if the organisation already has one) raises this substantially. Treat it as a storage backend behind the adapter, not as a place people browse manually - otherwise the metadata in your database drifts from reality.

4.4 A caveat on Firebase Storage

CAUTION — CONFIRM YOUR FIREBASE BILLING PLAN BEFORE DESIGNING AROUND IT

Google now requires the pay-as-you-go Blaze plan to provision Cloud Storage on newly created Firebase projects; the free Spark allowance of 5 GB applies to projects that already had storage enabled. Project ce0001 already exists and is referenced by the frontend as ce0001.firebasestorage.app, so it may qualify - but confirm in the Firebase console before relying on it. Even on Blaze the first 5 GB remains free, so the exposure is small; the point is to know which plan you are on.

4.5 Recommended stack

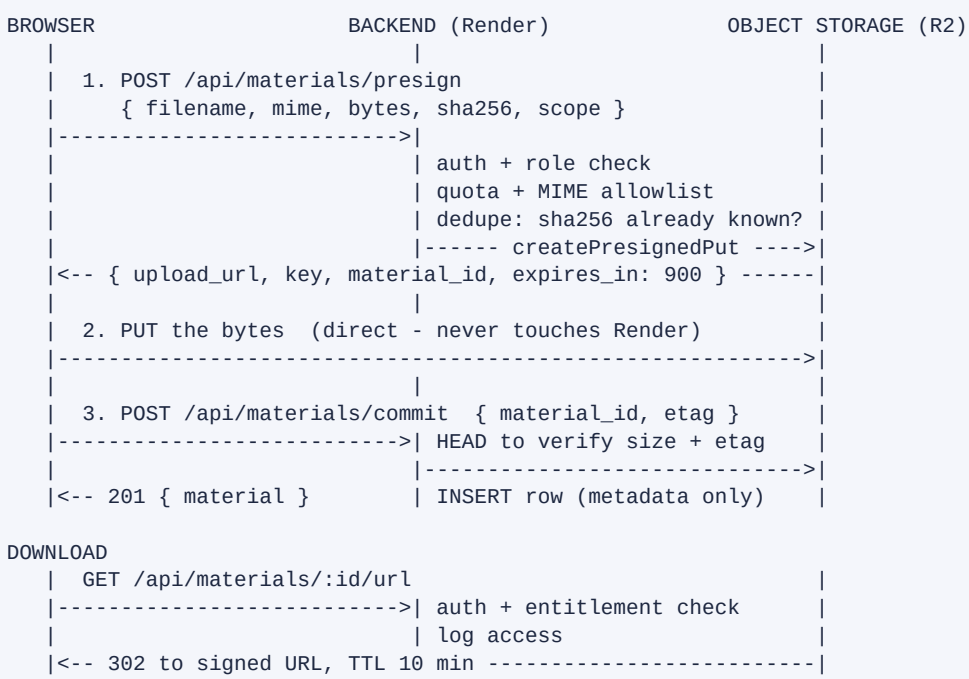
TIER	CONTENT	PLATFORM	WHY
Hot / small	PDFs, slides, scripts under 25 MB	Supabase Storage, then R2	One system for small files; no new credentials to start
Primary	Video, audio, large documents	Cloudflare R2	Zero egress, S3 API, 10 GB free
Delivery	Lecture video playback	YouTube unlisted, or Bunny Stream when budget allows	Free adaptive streaming and a global CDN you do not operate
Overflow	Bulk documents	Google Drive (15 GB)	Large free allowance, mature API
Archive	Closed batches, superseded versions	Storj (25 GB) or B2	Cheap cold storage; retrieval is rare

RECOMMENDED — START WITH TWO, DESIGN FOR FIVE

Begin with Supabase Storage for documents and Cloudflare R2 for media - that alone unblocks you. Because every provider sits behind the same adapter interface, adding Drive, Storj or YouTube later is a new file in one directory and a row in a config table, with no change to callers.

5. Target Architecture

5.1 Upload path - the browser talks to storage directly



DESIGN WHY THE REDIRECT MATTERS

Handing out a short-lived signed URL rather than proxying bytes through Express means a 2 GB video costs your dyno one database read and one redirect. Render never buffers the file, the 10 MB body limit becomes irrelevant, and requests cannot time out on large media. The entitlement check still happens on every request, so access remains controlled.

5.2 The provider adapter

One interface, one file per provider. Callers never import a vendor SDK directly, which is what makes the provider decision reversible.

```
// backend/storage/index.js
export const drivers = { r2, supabase, gdrive, b2, storj, youtube };
export function driverFor(provider) { return drivers[provider]; }

// every driver implements exactly this surface:
{
  name: 'r2',
  presignPut(key, { mime, bytes, expiresIn }) -> { url, headers },
  presignGet(key, { expiresIn, filename })    -> url,
  head(key)                                   -> { bytes, etag, mime } | null,
  delete(key)                                 -> void,
  copyTo(driver, key)                         -> key      // tier migration
}

// backend/storage/r2.js - R2 speaks S3, so this is the standard SDK
import { S3Client, PutObjectCommand, GetObjectCommand } from '@aws-sdk/client-s3';
import { getSignedUrl } from '@aws-sdk/s3-request-presigner';

const s3 = new S3Client({
  region: 'auto',
  endpoint: `https://${process.env.R2_ACCOUNT_ID}.r2.cloudflarestorage.com`,
  credentials: { accessKeyId: process.env.R2_ACCESS_KEY_ID,
                 secretAccessKey: process.env.R2_SECRET_ACCESS_KEY },
});
```

5.3 Content-addressed keys

Keys are derived from the file's SHA-256 hash, not from the batch or the upload time. Two consequences follow, both valuable: the same file uploaded twice is stored once, and a batch that reuses last term's material adds zero bytes.

```
key = `materials/${sha256.slice(0,2)}/${sha256.slice(2,4)}/${sha256}${ext}`

materials/9f/3c/9f3c8a1e...d4.mp4

// presign() first checks whether sha256 already exists:
//   found    -> skip the upload entirely, return the existing material_id
//   not found-> issue the pre-signed PUT
// Ten PD batches sharing a 15 GB library consume 15 GB, not 150 GB.
```

6. Database Schema Changes

Four new tables plus two small alterations. Nothing existing is dropped, so the change is additive and reversible.

```
-- 1. The material itself: one row per unique file, per course version
create table materials (
  id            uuid primary key default gen_random_uuid(),
  sha256        text not null unique,       -- content address = dedupe key
  provider      text not null,              -- r2 | supabase | gdrive | youtube | storj
  object_key    text not null,              -- or the YouTube video id
  filename      text not null,
  mime_type     text not null,
  bytes         bigint not null,
  kind          text not null,              -- document | video | audio | archive | code
  duration_secs int,                        -- media only
  domain        text,                       -- PD | DV | DFT
  course_version text,                      -- 'PD-2026.1'  <- NOT batch_no
  module_name   text,
  week_no       int,
  topic_name    text,
  visibility    text not null default 'batch',  -- batch | internal | public
  status        text not null default 'ready',  -- pending | ready | failed | archived
  uploaded_by   text not null,
  created_at    timestamptz default now()
);
create index on materials (domain, course_version, week_no);
create index on materials (kind, status);

-- 2. Which batches may see which material (many-to-many = store once, link many)
create table material_links (
  id           bigserial primary key,
  material_id  uuid not null references materials(id) on delete cascade,
  batch_no     text not null,
  release_at   timestamptz,                 -- null = immediately; else drip release
  released     boolean not null default false,
  linked_by    text not null,
  created_at   timestamptz default now(),
  unique (material_id, batch_no)
);
create index on material_links (batch_no, released, release_at);

-- 3. Who opened what, and when (entitlement audit + engagement signal)
create table material_access_log (
  id           bigserial primary key,
  material_id  uuid not null,
  batch_no     text,
  user_email   text not null,
  action       text not null,               -- url_issued | denied
  ip           inet,
  created_at   timestamptz default now()
);
create index on material_access_log (material_id, created_at);

-- 4. Provider registry: quotas and tiering policy live in data, not code
create table storage_providers (
  name           text primary key,          -- r2 | supabase | gdrive | storj | b2
  active         boolean default true,
  priority       int not null,              -- lowest number wins when routing
  max_bytes      bigint,                    -- the free-tier ceiling
  used_bytes     bigint default 0,          -- maintained on commit/delete
  accepts_kinds  text[] default '{}',       -- e.g. {video,audio}
  max_object_mb  int
);

-- 5. Alterations to the existing delivery table
alter table scheduled_emails add column attachment_material_id uuid references materials(id);
-- attachment_data is retained for now, backfilled to null, dropped after migration M0
```

DESIGN MATERIALS.COURSE_VERSION IS THE LOAD-BEARING COLUMN

Keying on course_version rather than batch_no is what makes the free tiers sufficient. material_links then answers "which batches can see this", and release_at turns the same table into the drip-release mechanism described in Section 9.

7. API Specification

7.1 New endpoints

METHOD + PATH	ROLE	PURPOSE
POST /api/materials/presign	admin, manager, coordinator, trainer	Validate, dedupe by sha256, return a pre-signed PUT URL
POST /api/materials/commit	same	Verify the object via HEAD, insert the metadata row
GET /api/materials	any authenticated	List materials visible to the caller; filters for batch, module, week, kind
GET /api/materials/:id/url	entitled users only	Log access, redirect to a short-lived signed URL
POST /api/materials/:id/link	admin, manager, coordinator	Attach a material to a batch, optionally with release_at
DELETE /api/materials/:id/link/:batch	admin, manager, coordinator	Revoke a batch's access without deleting the object
DELETE /api/materials/:id	admin	Soft-delete, then remove the object once no links remain
POST /api/materials/:id/archive	admin	Copy to the archive tier and repoint the row
GET /api/materials/quota	admin, manager	Per-provider used vs free-tier ceiling

7.2 Endpoints to retire

ENDPOINT	ACTION	REASON
POST /api/upload	Remove	Unauthenticated public file host; URLs die on redeploy (F3, F4)
app.use('/uploads', static)	Remove	Serves arbitrary uploaded content to anyone
multer({ dest: 'uploads/' })	Replace	Switch to memoryStorage with a size limit and MIME allowlist, for spreadsheet parsing only

7.3 Presign contract

```
POST /api/materials/presign        Authorization: Bearer <signed JWT>
{
  "filename":       "PD-W03-Physical-Design.mp4",
  "mime_type":      "video/mp4",
  "bytes":          734003200,
  "sha256":         "9f3c8a1e...d4",     // computed in the browser
  "kind":           "video",
  "domain":         "PD",
  "course_version": "PD-2026.1",
  "week_no":        3,
  "topic_name":     "Physical Design - Floorplanning"
}

200  already stored (dedupe hit - no upload needed)
{ "duplicate": true, "material_id": "7c1e...", "upload_url": null }

200  upload required
{
  "duplicate":   false,
  "material_id": "a93f...",
  "provider":    "r2",
  "upload_url":  "https://<acct>.r2.cloudflarestorage.com/...&X-Amz-Expires=900",
  "headers":     { "Content-Type": "video/mp4" },
  "expires_in":  900
}

413  the chosen provider has no room -> body names the next provider to try
415  mime_type not in the allowlist
```

7.4 Validation rules

- **MIME allowlist, not blocklist.** Permit pdf, office documents, images, mp4/webm, mp3/m4a, zip and plain text. Reject everything else, and never trust the client-supplied type alone - verify with a HEAD after upload.
- **Per-kind size ceilings.** Documents 50 MB, audio 500 MB, video 4 GB. Enforced at presign time, then confirmed against the real object size at commit.
- **Server-side hashing check.** Compare the committed object's etag or a recomputed digest against the declared sha256, so a client cannot claim a dedupe hit on a file it did not upload.
- **Entitlement on every read.** A learner may read a material only if a material_link exists for their batch and released is true. Never rely on the URL being secret.

8. Code Changes, File by File

FILE	CHANGE	TYPE	RISK
backend/storage/index.js	New. Driver registry and routing by kind, quota and priority.	New	Low
backend/storage/r2.js	New. S3 presign via @aws-sdk/client-s3 + s3-request-presigner.	New	Low
backend/storage/supabase.js	New. createSignedUploadUrl / createSignedUrl.	New	Low
backend/storage/gdrive.js	New. Resumable upload + permission handling (optional, later).	New	Medium
backend/storage/youtube.js	New. Metadata-only driver; playback by video id (optional).	New	Medium
backend/routes/materials.js	New. The nine endpoints from Section 7.	New	Low
backend/index.js:137	Replace multer disk storage with memoryStorage + limits + MIME filter.	Modify	Low
backend/index.js:6938-6950	Delete /api/upload and the static /uploads mount.	Delete	Medium
backend/index.js:9443-9503	mock-interview-feedback: upload once, link to batch, email a link.	Modify	Medium
backend/index.js:9622+	internal/feedback-share: same treatment for its attachment path.	Modify	Medium
backend/index.js:9878-9884	Dispatcher: resolve attachment_material_id; attach if small, else link.	Modify	High
backend/routes/coursePlanner.js	Persist generated xlsx/csv to storage instead of uploads/.	Modify	Medium
backend/email-worker.js	Delete (see companion report B1). Otherwise fix attachement_* typo.	Delete	Low
frontend/src/pages/Materials.js	New. Upload with progress, browse, link to batch, set release date.	New	Low
frontend/src/utils/upload.js	New. sha256 via SubtleCrypto, presign, direct PUT, commit.	New	Low
frontend/src/pages/TrainerDashboard.js	Add a per-topic materials panel.	Modify	Low
migrations/002_materials.sql	New. The DDL from Section 6.	New	Low

8.1 The one high-risk change

Only the dispatcher edit carries real risk, because it sits on the path that emails learners. It should be written to degrade safely: if a material cannot be resolved, fall back to the existing attachment_data when present, and otherwise send the message body without the attachment rather than failing the send.

```
// backend/index.js - inside the per-minute dispatcher, replacing lines 9878-9884
const attachments = [];
let linkHtml = '';

if (email.attachment_material_id) {
  const mat = await getMaterial(email.attachment_material_id);
  if (mat && mat.bytes <= INLINE_LIMIT) {              // INLINE_LIMIT = 8 MB
    attachments.push({ filename: mat.filename,
                       content: await fetchObject(mat) });
  } else if (mat) {
    const url = await signedUrl(mat, { expiresIn: 7 * 24 * 3600 });
    linkHtml = `<p><a href="${url}">Download ${mat.filename}</a>` +
               ` (${humanSize(mat.bytes)}, link valid 7 days)</p>`;
  }
} else if (email.attachment_name && email.attachment_data) {
  attachments.push({ filename: email.attachment_name,   // legacy rows, during migration
                     content: Buffer.from(email.attachment_data, 'base64') });
}

await sendRawEmail({ ..., html: email.body_html + linkHtml, attachments });
```

CAUTION MAILJET CAPS A MESSAGE AT ROUGHLY 15 MB

Attaching real course material was never going to work regardless of where it is stored. The 8 MB inline threshold above keeps small feedback PDFs behaving exactly as they do today, while anything larger automatically becomes a link. That is a behaviour change learners will notice, so it belongs in the release note.

9. Integration With the One-Trigger Platform

This section connects to the companion document, AUTOMATION_BLUEPRINT.pdf. Materials become another thing the launch trigger arranges, rather than a separate manual chore.

9.1 A new stage in the graph

```
LAUNCH
  +-- validate_charter
  +-- allocate_classroom / check_licenses
  +-- import_learners
  +-- generate_planner  ->  ingest_planner
  +-- assign_trainers
  |
  +== link_materials   <<< NEW >>>
  |     charter names a course_version, e.g. 'PD-2026.1'
  |     SELECT id FROM materials WHERE course_version = $1
  |     INSERT INTO material_links (material_id, batch_no, release_at)
  |       release_at = the planner date of the matching week/topic
  |     -> zero bytes copied. Ten batches, one library.
  |
  +== derive_schedule
  |     ... existing reminder rules ...
  |     + for each material_link with a release_at:
  |         emit task material_release  (run_after = release_at)
  |
  +-- schedule_welcome_emails
  v
RUNNING -> (reconciler tick) -> CLOSING -> CLOSED
                                  |
                                  +-- archive_materials: move batch-specific
                                      recordings to the cold tier; course
                                      library untouched and reused next term
```

9.2 Drip release, driven by the planner

Because release_at is copied from the course planner, materials unlock as the course actually progresses. Week 3 slides appear the morning week 3 is taught. If a topic is rescheduled - which the system already tracks through actual_date and date_change_audit - the re-derive step moves the release with it, and no one has to remember.

```
-- the reconciler handler for a material_release task
UPDATE material_links SET released = true
 WHERE batch_no = $1 AND release_at <= now() AND released = false
RETURNING material_id;

-- then, through the existing dispatch() fan-out:
dispatch({
  kind: 'materials_released', batch_no,
  audience: 'learners', channel: 'both',        -- email + push
  template_name: 'Materials Available',
  vars: { week_no, topic_name, count, portal_url }
});
```

RECOMMENDED NO NEW DELIVERY MACHINERY

material_release is an ordinary task, and the notification is an ordinary dispatch. It reuses the scheduled_emails queue, the retry logic and the push fan-out that already run in production. The materials feature adds a table and a handler, not a second delivery system.

9.3 New automation rules this unlocks

TRIGGER	OFFSET	AUDIENCE	ACTION
Topic taught (planner date)	+0 at 18:00	Learners	Release that topic's materials, email and push the links
Weekly Quiz scheduled	-2 days	Learners	Release the revision pack for that week
Intermediate Assessment	-7 days	Learners	Release the consolidated revision bundle
Trainer assigned to a module	+0	Trainer	Grant access to that module's lab material and answer keys
Material added to a live course	+0	Linked batches	Notify only batches past that week
Course closure	+1 day	Learners	Issue a long-lived archive bundle link, then revoke batch access
Material never opened	+7 days	Coordinator	Engagement flag for follow-up

9.4 Where the agentic layer helps

- **Auto-classification on upload.** A trainer uploads 40 files named PD_wk3_v2_final.pdf. An agent proposes domain, week, module and topic by matching against course_planner_data, and a human confirms in one click rather than filling 40 forms.
- **Quota steward.** A nightly agent watches used_bytes against each provider ceiling, identifies cold or superseded material, and proposes archive moves before a tier fills.
- **Duplicate and drift detection.** Near-identical files under different names, or a course_version whose materials no longer match the planner's topics.
- **Accessibility and search.** Transcribe lecture audio to build a searchable index and auto-generate captions - a genuinely useful application of a model, entirely off the critical path.

10. Migration Plan

M0 - Stop the bleeding (half a day)

- Add attachment_material_id to scheduled_emails; leave attachment_data in place for now
- Change mock-interview-feedback to store the file once and reference it, instead of base64 per learner
- Backfill: null out attachment_data on rows already sent, then VACUUM FULL to reclaim the space
- Expected result: the largest single consumer of database storage disappears immediately

M1 - Storage adapter (1-2 days)

- Create the Cloudflare R2 bucket and credentials; add the four R2_* environment variables
- Implement backend/storage/ with the r2 and supabase drivers
- Apply migrations/002_materials.sql
- Seed storage_providers with the free-tier ceilings from Section 4

M2 - Materials API and UI (2-3 days)

- Implement backend/routes/materials.js - presign, commit, list, url, link, delete
- Build the frontend upload utility (SubtleCrypto hashing, direct PUT, progress) and the Materials page
- Remove /api/upload and the static /uploads mount; switch multer to memoryStorage with limits
- Migrate the eight committed files in backend/uploads/ into storage, then delete them from git

M3 - Orchestrator integration (2 days)

- Add the link_materials stage and the material_release task handler
- Extend derive_schedule to emit release tasks from planner dates
- Add the materials automation_rules from 9.3
- Add a materials panel to the trainer and learner views

M4 - Lifecycle (1-2 days)

- Archive stage on course closure; add the storj or b2 driver as the cold tier
- Quota dashboard at /api/materials/quota
- Retention policy: superseded course versions move to archive after two terms

Total: approximately one week of engineering. M0 alone recovers most of the database pressure and can ship on its own, before any provider decision is finalised.

11. Costs Beyond the Free Tier

Indicative monthly figures for when the libraries outgrow free allowances. Egress assumes six batches of 25 learners per year.

PROVIDER	100 GB	500 GB	1 TB	EGRESS
Cloudflare R2	~1.35 USD	~6.75 USD	~13.50 USD	Free - the decisive advantage
Backblaze B2	~0.60 USD	~3.00 USD	~6.00 USD	Free up to 3x stored, then ~0.01/GB
Storj DCS	~0.40 USD	~2.00 USD	~4.00 USD	~0.007/GB
Wasabi	~6.99 USD (1 TB floor)	~6.99 USD	~6.99 USD	Free, but a 1 TB minimum applies
AWS S3 Standard	~2.30 USD	~11.50 USD	~23.00 USD	~0.09/GB - the expensive line
Bunny Stream (video)	~0.50 USD	~2.50 USD	~5.00 USD	~0.01/GB with token auth

RECOMMENDED THE REALISTIC BILL IS SINGLE-DIGIT DOLLARS

At 45-60 GB of course material with video delivered by YouTube or Bunny, the expected steady-state cost is roughly 1 to 5 USD per month even after leaving the free tiers entirely. The engineering effort is worth doing for correctness and reliability; the cost saving is a secondary benefit.

12. Risks, Limits and Caveats

RISK	MITIGATION
Free tiers change or are withdrawn	The adapter interface makes a provider swap a one-file change. Keep a second driver configured and the archive tier populated so a migration is a copy, not a rebuild.
YouTube unlisted is not access control	Treat it as public. Never host answer keys, assessment papers or anything commercially sensitive there. Use R2 signed URLs for those.
Signed URLs get forwarded	Keep TTLs short (10 minutes for viewing, 7 days for email links), log every issuance in material_access_log, and make links single-batch so leaks are traceable.
Firebase Spark vs Blaze uncertainty	Confirm the plan for project ce0001 before relying on the 5 GB allowance. The exposure is small either way.
Client-declared sha256 is spoofable	Verify with a HEAD on commit and reject mismatches; never mark a material ready on the client's word alone.
Large uploads fail midway	Use multipart or resumable uploads for files over 100 MB; materials stay in status pending until commit succeeds, and a nightly sweep deletes orphans.
Copyright and licensing of third-party material	Record a licence field per material and keep vendor PDFs in an internal-visibility tier that is never linked to learner batches.
Deleting a material still linked elsewhere	Deletion is soft; the object is removed only once no material_links remain, which content addressing makes safe.

13. Appendix - Evidence Index

FINDING	LOCATION
Base64 attachment written per learner	backend/index.js:9462, 9469-9489
Dispatcher decodes attachment_data	backend/index.js:9878-9884
Worker reads misspelled attachement_*	backend/email-worker.js:75-79
multer disk storage, no limits	backend/index.js:137
Public upload endpoint	backend/index.js:6938-6947
Static /uploads mount	backend/index.js:6950
JSON body limit of 10 MB	backend/index.js:126
Supabase Storage never used	repo-wide grep for supabase.storage - no matches
Firebase Admin initialised	backend/push/firebaseAdmin.js:38-59
Firebase bucket already configured	frontend/src/push/webPush.js (ce0001.firebasestorage.app)
Upload artefacts committed to git	backend/uploads/ - 8 files, 299 KB
Planner artefacts on ephemeral disk	backend/routes/coursePlanner.js:21 (WORK_DIR)
Course planner drives all dates	course_planner_data - see AUTOMATION_BLUEPRINT.pdf section 5

Scope note

This analysis was read-only; no application file was modified. Free-tier figures are as published by each vendor at the time of writing and should be verified before commitment. Pricing in Section 11 is indicative and excludes taxes and per-operation charges, which are negligible at this scale.